

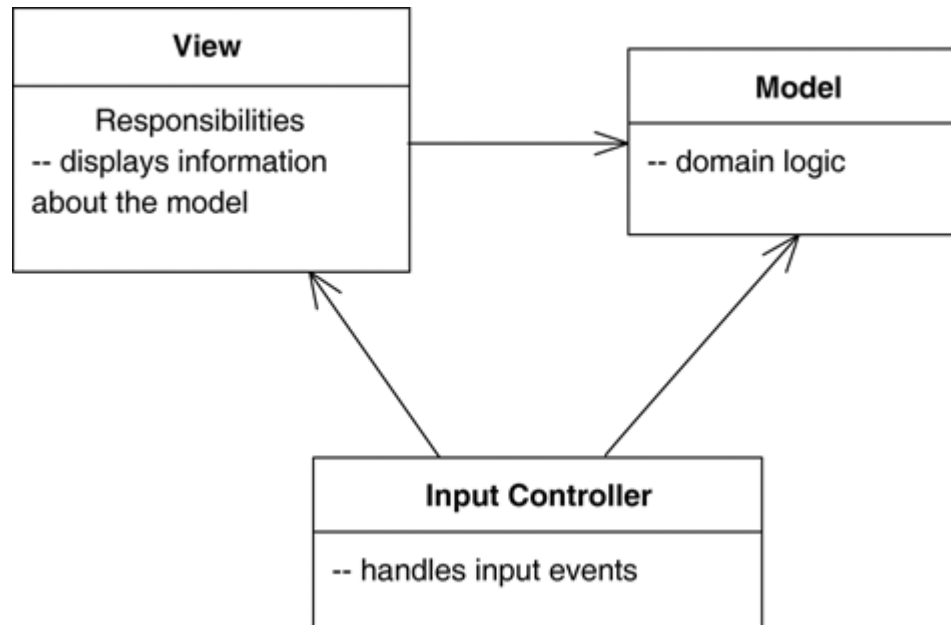


UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Class Diagrams - Advanced

Francesco Guerra
francesco.guerra@unimore.it

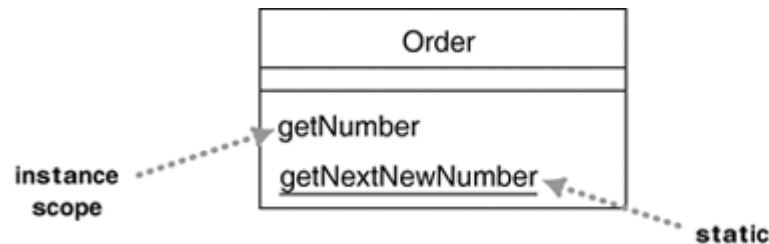
Responsibilities



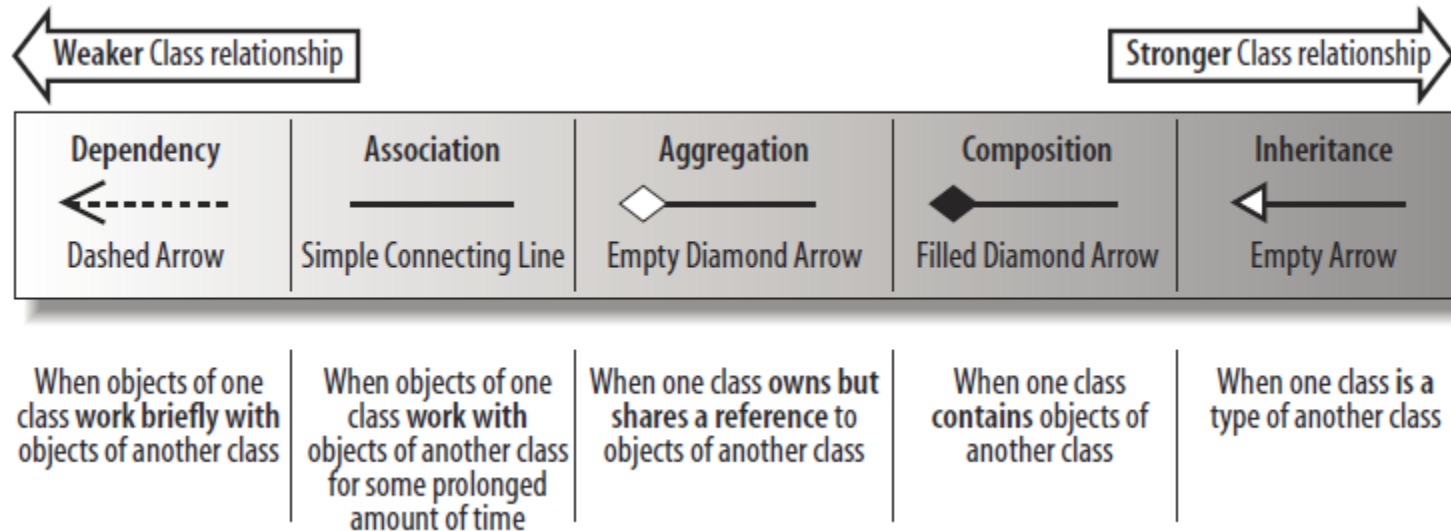
- ▶ The best way to show them is as comment strings in their own compartment in the class.
- ▶ You can name the compartment, but there's rarely any potential for confusion

Static Operations and Attributes

- ▶ The UML refers to an operation or an attribute that applies to a class rather than to an instance as static
 - ▶ Static features are underlined on a class diagram

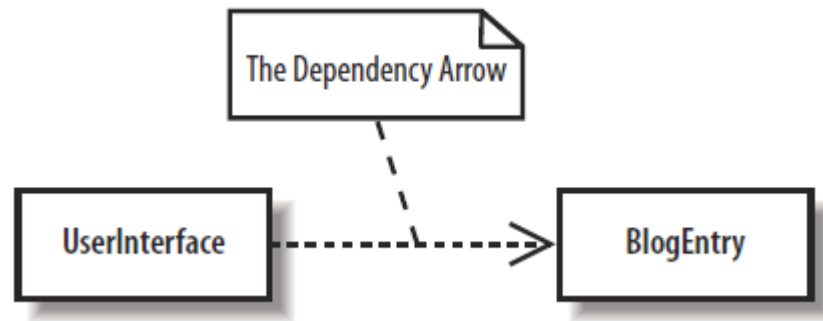


Class Relationships



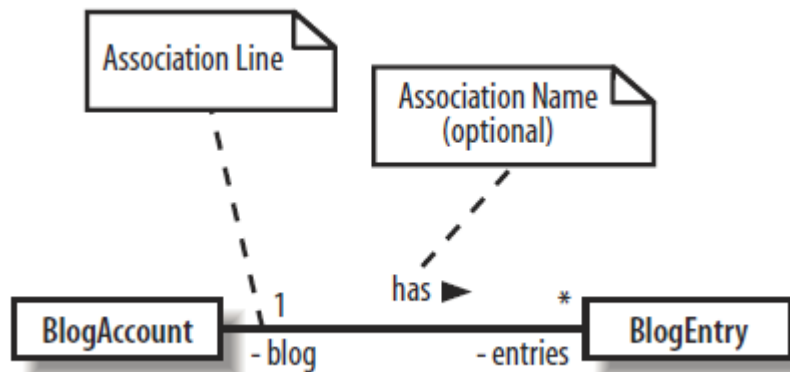
- ▶ Classes do not live in a vacuum—they work together using different types of relationships.

Dependency



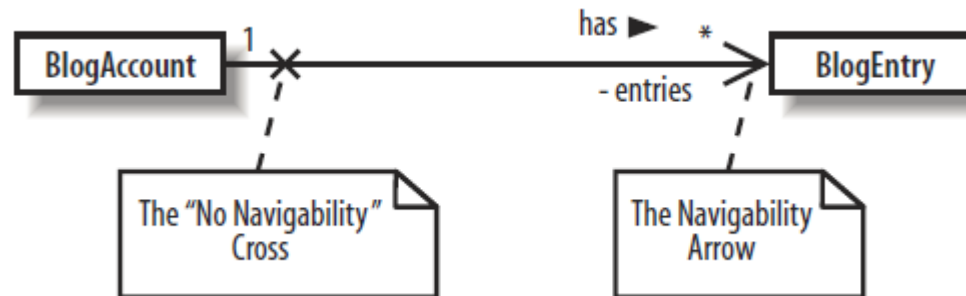
- ▶ Already introduced.
- ▶ The UserInterface and BlogEntry classes simply work together at the times when the user interface wants to display the contents of a blog entry.
 - ▶ The two classes of object are dependent on each other to ensure they work together at runtime.
- ▶ A dependency implies only that objects of a class *can* work together
- ▶ It is considered to be the weakest direct relationship that can exist between two classes.
- ▶ The dependency relationship is often used when you have a class that is providing a set of general-purpose utility functions, such as in Java's mathematics (java.math) packages. Classes depend java.math classes to use the utilities that those classes offer.

Association



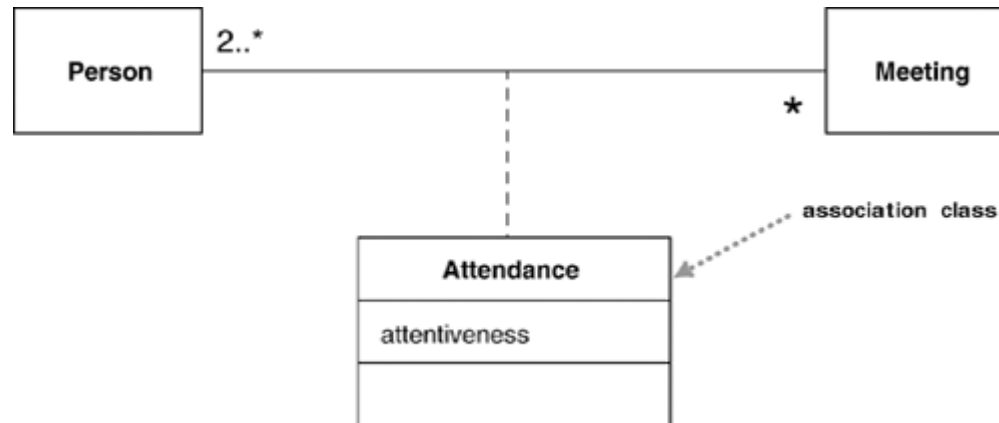
- ▶ Already introduced.
- ▶ *Association* means that a class actually contains a reference to an object, or objects, of the other class in the form of an attribute.
 - ▶ If you find yourself saying that a class *works with* an object of another class, then the relationship between those classes is a great candidate for association rather than just a dependency.
- ▶ *Navigability* is often applied to an association relationship to describe which class contains the attribute that supports the relationship.
- ▶ Without more information about the association between the BlogAccount and BlogEntry classes, it is impossible to decide which class should contain the association introduced attribute;
 - ▶ In this case, both classes have an attribute added.
 - ▶ It is more common to have only one class referencing the other in an association.

Association (2)



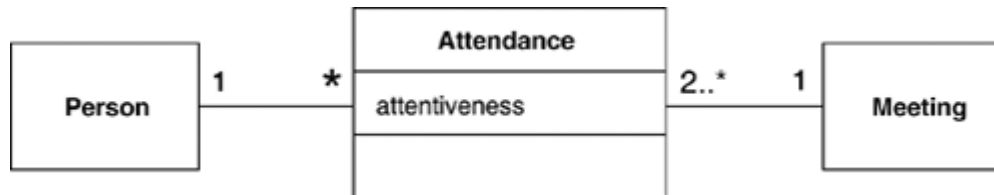
- ▶ In our system, it makes more sense to be able to ask a blog account what entries it contains, rather than asking the entry what blog account it belongs to.
- ▶ In this case, we use navigability to ensure that the **BlogAccount** class gets the association introduced attribute.

Association Class



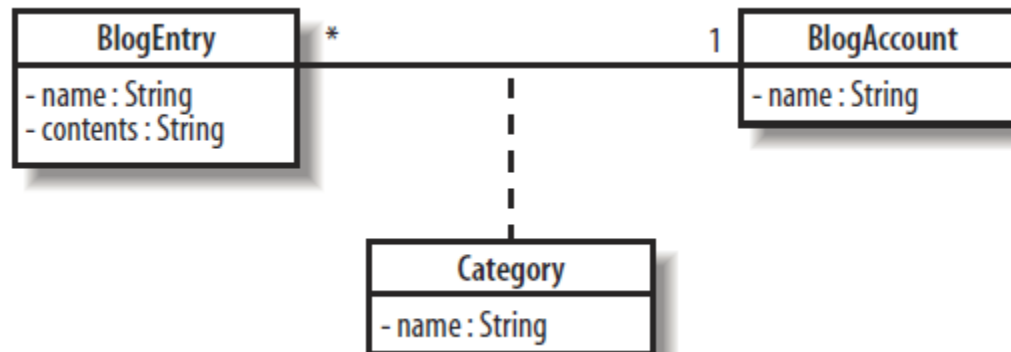
- ▶ Association classes allow you to add attributes, operations, and other features to associations.
- ▶ We can see from the diagram that a person may attend many meetings.
 - ▶ We need to keep information about how awake that person was;
 - ▶ We can do this by adding the attribute **attentiveness** to the association.
- ▶ The association class adds an extra constraint, in that there can be only one instance of the association class between any two participating objects

Association Class (2)



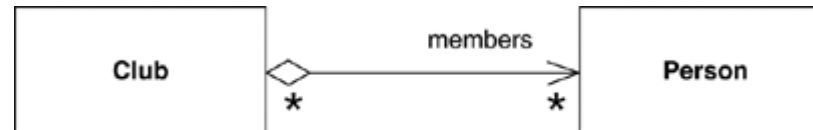
- ▶ Implement an association class as if it were a full class but to provide methods that get information to the classes linked by the association class
- ▶ class Person
 - List getAttendances()
 - List getMeetings()
- ▶ Note that with this representation a person can attend more times to the same meeting

Association Class (3)

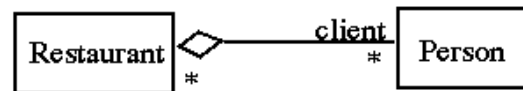


- ▶ Association classes are particularly useful in complex cases when you want to show that a class is related to two classes *because* those two classes have a relationship with each other.
 - ▶ *BlogEntry is associated with a Category by virtue of the fact that it is associated with a particular BlogAccount*

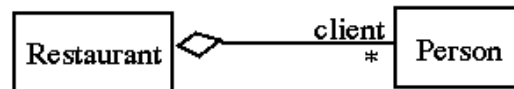
Aggregation



- ▶ Aggregation is the part-of relationship
 - ▶ The difficult thing is considering what the difference is between aggregation and association



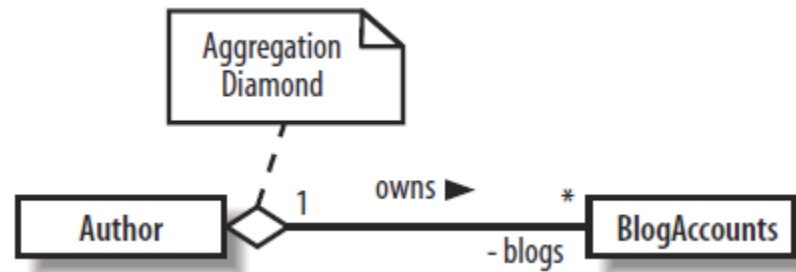
Each Person eats at any number of Restaurants



Each Person eats at one Restaurant.

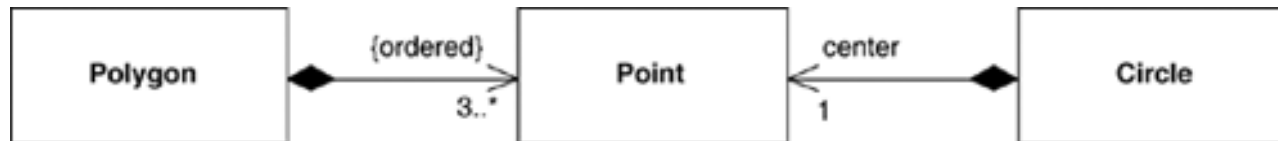
- ▶ Aggregation is really just a stronger version of association and indicates that a class actually *owns but may share* objects of another class.
- ▶ Furthermore, there is hardly a difference between aggregations and associations during implementation

Aggregation (2)



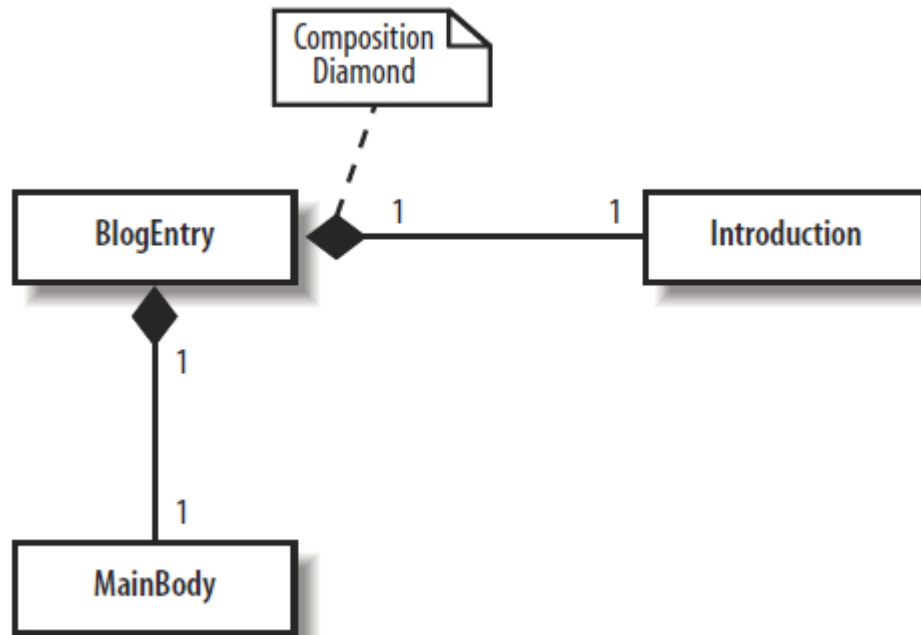
- ▶ The relationship between an author and his blogs is much stronger than just association.
- ▶ An author owns his blogs, and even though he *might* share them with other authors, in the end, his blogs are his own, and if he decides to remove one of his blogs, then he can!

Composition



- ▶ It is a form of aggregation
- ▶ The general rule is that, although a class may be a component of many other classes, any instance must be a component of only one owner.
- ▶ The class diagram may show multiple classes of potential owners, but any instance has only a single object as its owner
- ▶ Another assumption is that if you delete the polygon, it should automatically ensure that any owned Points also are deleted.
- ▶ Composition is a good way of showing properties that own by value, properties to value objects or properties that have a strong and somewhat exclusive ownership of particular other components.

Composition (2)



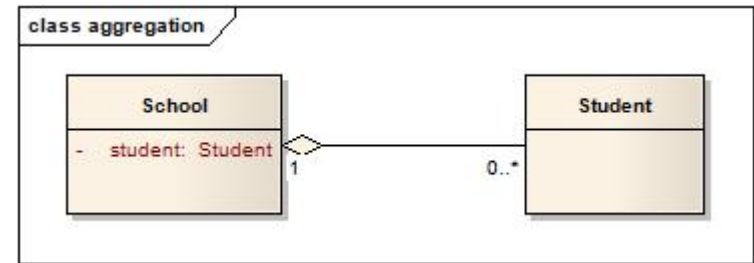
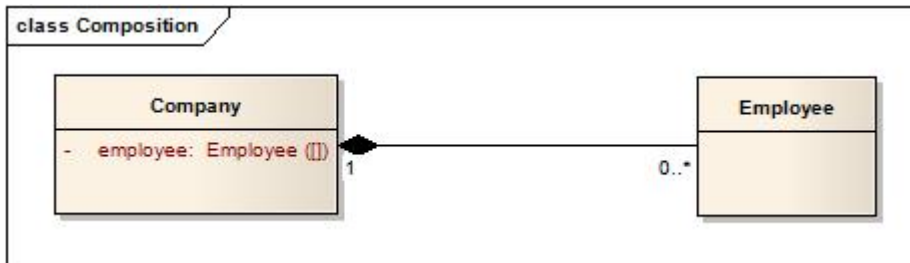
- ▶ A blog entry's introduction and main body sections are actually *parts of* the blog entry itself and won't usually be shared with other parts of the system.
- ▶ If the blog entry is deleted, then its corresponding parts are also deleted.
- ▶ This is exactly what composition is all about: you are modeling the internal parts that make up a class.

Aggregation vs. Composition

- ▶ Composition relationship :
 - ▶ When attempting to represent real-world whole-part relationships, e.g., an engine is a part of a car.
- ▶ Aggregation relationship :
 - ▶ When representing a software or database relationship, e.g., car model engine ENG01 is part of a car model CM01, as the engine, ENG01, may be also part of a different car model.
- ▶ Thus the aggregation relationship is often "catalog" containment to distinguish it from composition's "physical" containment.

(from Wikipedia)

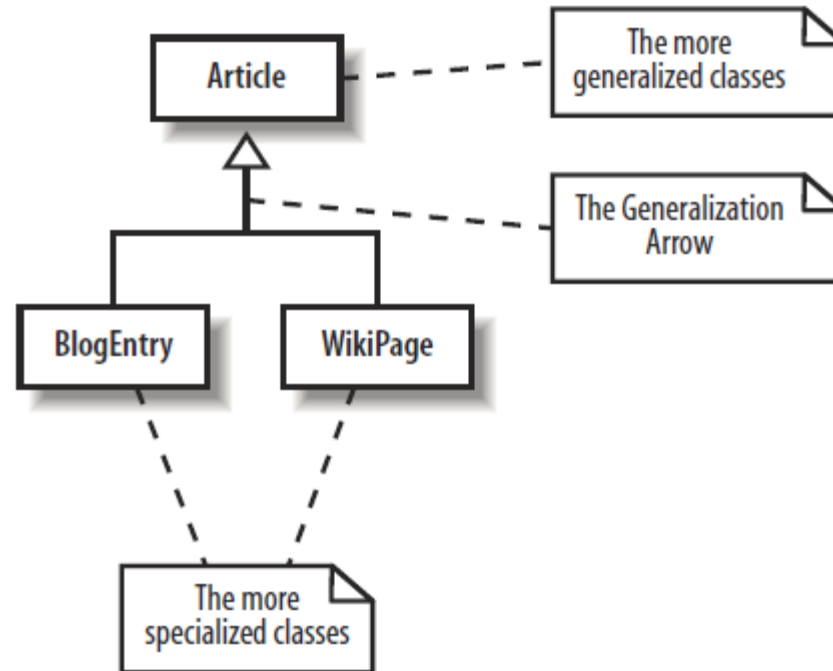
Example in java



```
public class Employee {  
}  
  
public class Company {  
    private Employee[] employee;  
}
```

```
public class Student {  
}  
  
public class School {  
    private Student[] student;
```

Classification and Generalization

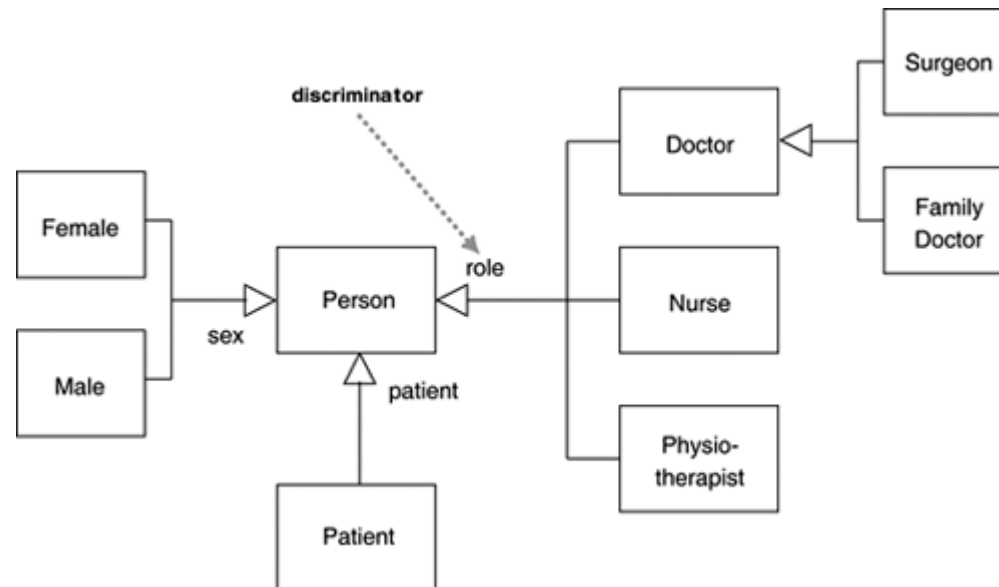


- ▶ The UML uses the generalization symbol to show generalization.
- ▶ If you need to show classification, use a dependency with the «instantiate» keyword.

Generalization

- ▶ The key to why inheritance is called generalization in UML is in the difference between what a parent class and a child class each represents.
 - ▶ Parent classes describe a more *general* type, which is then made more specialized in child classes.
- ▶ A child class inherits and reuses all of the attributes and methods that the parent contains and that have public, protected, or default visibility.
 - ▶ So, generalization offers a great way of expressing that one class is a type of another class, and it offers a way of reusing attributes and behavior between the two classes.
 - ▶ Nevertheless, since a child class can see most of the internals of its parent, it becomes tightly coupled to its parent's implementation.
- ▶ One of the principles of good object-oriented design is to avoid tightly coupling classes.
- ▶ Generalization is the strongest form of class relationship because it creates a tight coupling between classes.
- ▶ Therefore, it's a good rule of thumb to use generalization only when a class really is a more specialized type of another class and not just as a convenience to support reuse.

Multiple Generalization



- ▶ If you use multiple classification, you need to be sure that you make it clear which combinations are legal.
 - ▶ Generalization sets are by default disjoint: Any instance of the supertype may be an instance of only one of the subtypes within that set
 - ▶ The combination (Patient, Doctor, Nurse) is illegal because it contains two types from the role generalization set.

Derived Properties



- ▶ Derived properties can be calculated based on other values.
 - ▶ You can use derivation to indicate the difference between a calculated value and a stored value.
- ▶ It indicates a constraint between values.
 - ▶ In this case, we are saying that the constraint among the three values holds, but it isn't important which of the three values is computed.
- ▶ Derivation can also be applied to properties using association notation. In this case, you simply mark the name with a /.

Keywords

- ▶ UML often tries to reduce the number of symbols and use keywords instead. If you find that you need a modeling construct that isn't in the UML but is similar to something that is, use the symbol of the existing UML construct but mark it with a keyword to show that you have something different
- ▶ Keywords are usually shown as text between guillemets or in other cases in curly brackets.
- ▶ Some keywords are so common that they often get abbreviated: «interface» often gets abbreviated to «I» and {abstract} to {A}.
 - ▶ Such abbreviations are very useful, particularly on whiteboards, but nonstandard, so if you use them, make sure you find a spot to spell out what they mean.

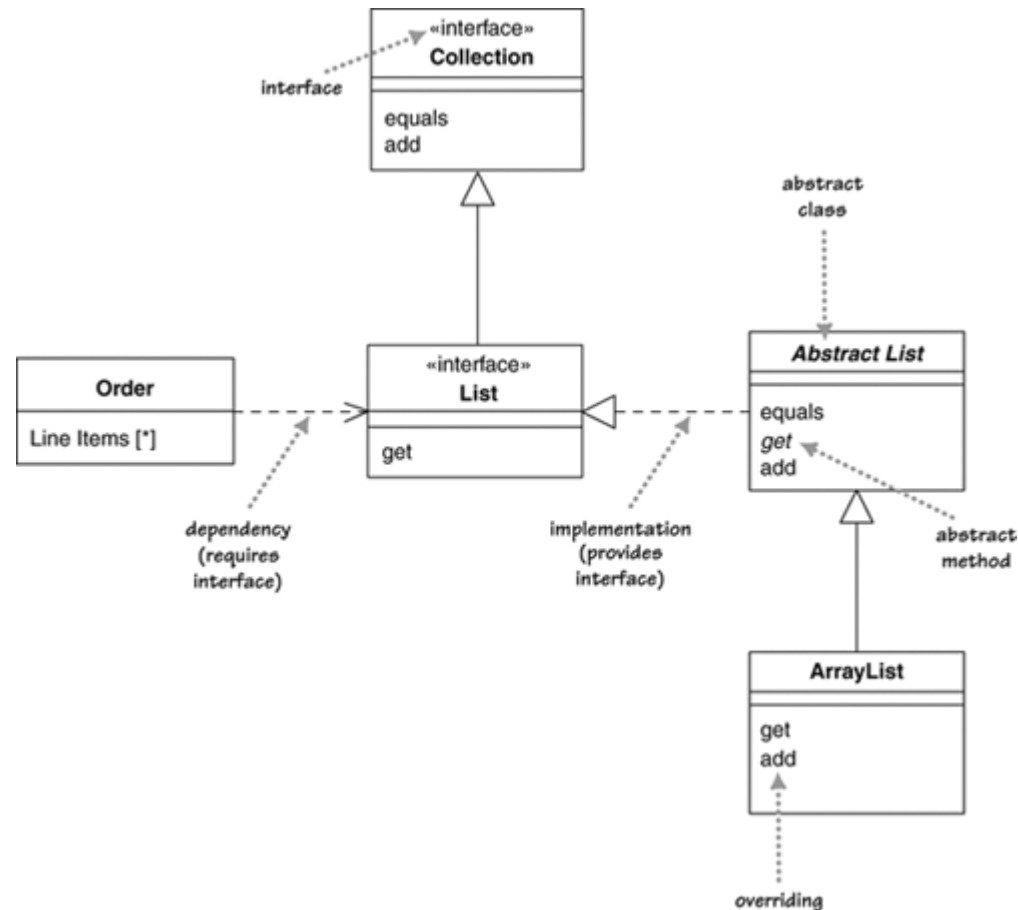
Abstract Classes vs. Interfaces

- ▶ Classes that cannot be instantiated themselves are modeled as *abstract classes*.
 - ▶ only their subclasses can be instantiated.
 - ▶ The common way to indicate an abstract class or operation is to italicize the name.
 - ▶ You can also make properties abstract. Italics are tricky to do on a whiteboards, so you can use the label: {abstract}.
- ▶ Abstract classes are used exclusively to highlight common characteristics of their subclasses and are therefore only useful in the context of generalization relationships.
 - ▶ Operations of abstract classes can also be labeled as abstract.
 - ▶ An *abstract operation* does not offer any implementation itself.
 - ▶ it requires an implementation in the concrete subclasses.

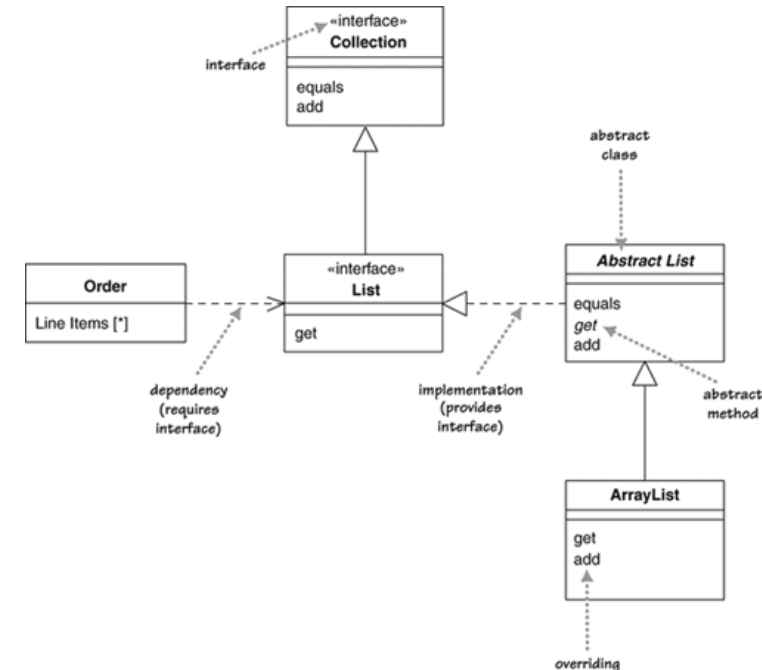
Interfaces and Abstract Classes

- ▶ An *interface* does not have an implementation or any direct instances.
 - ▶ An interface represents a contract.
 - ▶ mark an interface with the keyword «interface».
 - ▶ Operations of interfaces never have an implementation.
- ▶ The classes that enter into this contract, that is, the classes that implement the interface, obligate themselves to provide the behavior specified by the interface.
- ▶ An “is a” relationship between an interface and the classes that implement it is not necessary.
- ▶ Classes have two kinds of relationships with interfaces: providing and requiring.
 - ▶ A class provides an interface if it is substitutable for the interface.
 - ▶ In Java a class can do that by implementing the interface.
 - ▶ A class requires an interface if it needs an instance of that interface in order to work.
 - ▶ Essentially, this is having a dependency on the interface

Interfaces and Abstract Classes



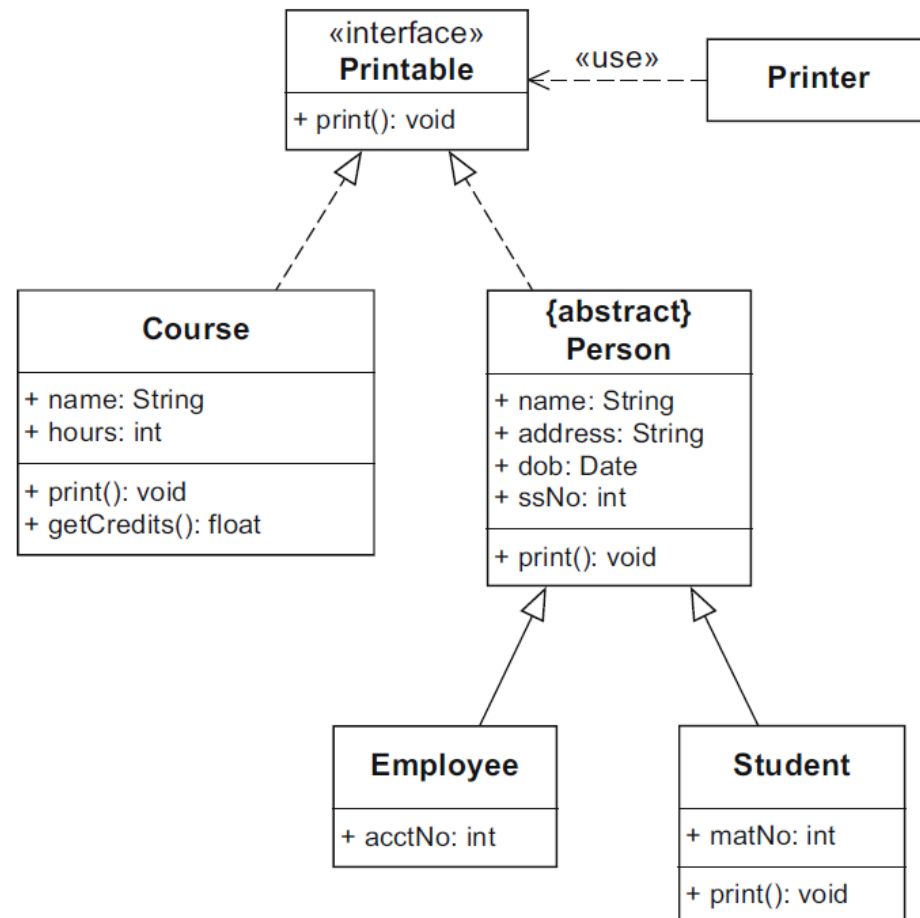
Interfaces and Abstract Classes (3)



- ▶ The ArrayList itself is a subclass of the AbstractList class.
- ▶ AbstractList provides some implementation of the List behavior. In particular, the get method is abstract.
- ▶ ArrayList implements get but also overrides some of the other operations on AbstractList.

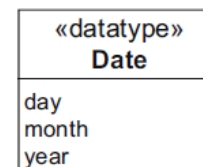
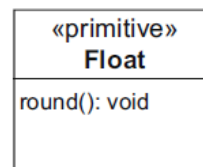
Interfaces and Abstract Classes (4)

- ▶ Why don't I simply avoid this and have Order use ArrayList directly?
 - ▶ By using the interface, I allow myself the advantage of making it easier to change implementations later on if I need to.
 - ▶ Another implementation may provide performance improvements, some database interaction features, or other benefits.
 - ▶ By programming to the interface rather than to the implementation, I avoid having to change all the code should I need a different implementation of List.



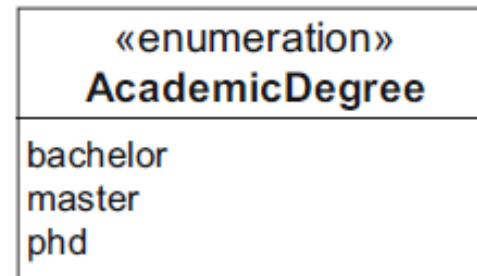
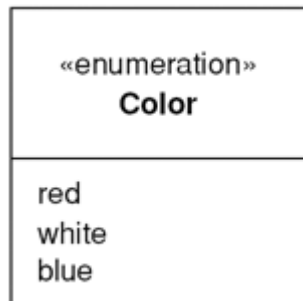
Data types

- ▶ Attributes, parameters, and return values of operations have a type that *Class* vs. *data type* specifies which concrete forms they may take.
 - ▶ The name of a person has the type String.
- ▶ A type can be either a class or a *data type*.
 - ▶ Instances of data types are also referred to as their *values*.
 - ▶ In contrast to instances of classes (objects), values do not have their own identity.
 - ▶ If two values are identical, they cannot be differentiated.
- ▶ In UML, a data type is visualized in the same way as a class, with the *Data type* difference that the name of the data type is annotated with the additional keyword «datatype».
- ▶ *Primitive data types* do not have any internal structure.
 - ▶ In UML there are four pre-defined primitive data types: Boolean, Integer, UnlimitedNatural, and String.
- ▶ User-defined primitive data types are identified by the specification of the keyword «primitive».



Enumerations

- ▶ *Enumerations* are data types whose values are defined in a list.
- ▶ Enumerations are used to show a fixed set of values that don't have any properties other than their symbolic value.
- ▶ They are shown as the class with the «enumeration» keyword.



Esercizio

- ▶ Una cantina è formata da più locali, ogni stanza contiene diversi scaffali e su questi sono collocate bottiglie di vini diversi. Per questione di luce e aereazione, le bottiglie sono periodicamente spostate da una stanza all'altra, ma occorre tenere traccia del giorno in cui una certa bottiglia è stata messa in una stanza per gestire il processo di invecchiamento.
- ▶ Le bottiglie dello stesso tipo di vino in uno scaffale devono essere facilmente identificabili.



References

- ▶ Martin Fowler, UML Distilled: A Brief Guide to the Standard Object Modeling Language , Pearson Addison Wesley
- ▶ Russ Miles, Kim Hamilton: Learning UML 2.0 A Pragmatic Introduction to UML, O'Reilly Media, 2006
- ▶ <https://dzone.com/articles/uml2-class-diagram-java>
- ▶ Martina Seidl, Marion Scholz, Christian Huemer, Gerti Kappel: UML @ Classroom - An Introduction to Object-Oriented Modeling, Springer 2015

MATERIALE DIDATTICO INTERNO DEL CORSO DI Ingegneria del Software.

La diffusione e pubblicazione on line del materiale è assolutamente proibita.